Chapter 1

The Landscape of Programming and C++

1.1 What Is Programming?

Programming is the act of constructing a sequence of instructions that a computing machine can execute to solve a specific problem or perform a defined task. Those instructions are written in a programming language, which provides a structured, human-readable notation for expressing computations. The programmer's job is to translate a real-world problem into an algorithmic form, then encode that algorithm in the syntax and semantics of the chosen language. The result is a program, a static artifact that, when run, becomes a dynamic process manipulating data, interacting with input and output devices, and producing useful results.

The fundamental challenge of programming is managing complexity. Every nontrivial program involves thousands or millions of individual operations, and the human mind cannot hold all those details simultaneously. Programming languages, and the techniques for using them, are therefore designed to provide abstraction mechanisms: ways to name compound operations, to encapsulate data with the functions that operate on it, to build hierarchies of types, and to reuse generic patterns. Abstraction is not merely a convenience; it is a cognitive necessity for building large, correct, maintainable systems.

1.2 What Is C++?

C++ is a general-purpose programming language created by Bjarne Stroustrup, first released in 1985. It originated as an extension of the C language, adding object-oriented features such as classes, inheritance, and polymorphism, along with stronger type checking. Over the decades it has evolved into a multi-paradigm language supporting procedural, object-oriented, generic, and functional programming styles. Modern C++, standardized in 2011 and subsequently updated in 2014, 2017, 2020, and 2023, bears little resemblance to the early versions. It provides high-level abstractions with zero-overhead cost, direct hardware manipulation, and deterministic control over resources, all within a single coherent language.

The defining philosophy of C++ is that you should not pay for what you do not use, and you should be able to express your intent directly to the compiler, enabling aggressive optimization. This makes C++ the language of choice for systems where performance, latency, and resource consumption are critical: operating systems, game engines, web browsers, financial trading systems, embedded firmware, and high-performance scientific computing. At the same time, C++ offers enough abstraction capability to build large-scale applications with complex architectures.

1.3 Historical Context

To understand C++, one must understand its relationship with C. C was designed in the early 1970s by Dennis Ritchie for systems programming at Bell Labs. It provided a thin abstraction over assembly language, with manual memory management, pointers, and a

minimal runtime. C became the language for writing Unix and most subsequent operating system kernels, device drivers, and embedded software.

Stroustrup began working on "C with Classes" in 1979, driven by the need to combine C's efficiency with the modularity and code organization benefits of Simula's class model. The first commercial release of C++ appeared in 1985. The language added classes, derived classes, strong type checking, inlining, and default arguments. Over the next decade, templates, exceptions, namespaces, and the Standard Template Library (STL) were incorporated. The first ISO standard, C++98, was published in 1998, followed by a minor update in 2003.

The 2011 standard, known as C++11, was a transformative release. It introduced move semantics, lambda expressions, automatic type deduction, range-based for loops, smart pointers, threading support, and much more. It fundamentally changed how C++ is written, making the language feel almost new. C++14 and C++17 refined these features and added improvements like generic lambdas, structured bindings, and filesystem support. C++20 added concepts, modules, ranges, coroutines, and consteval. C++23 continued the evolution with further refinements. Today, "modern C++" refers collectively to the style and feature set introduced since C++11.

1.4 Where C++ Is Used

C++ dominates domains where performance is paramount. Operating system kernels, device drivers, and firmware are written in C and C++ because they require direct hardware access, deterministic timing, and minimal runtime overhead. Game engines like Unreal Engine use C++ for the core simulation loop, physics, rendering, and memory management. Web browsers such as Chrome and Firefox use C++ for their rendering engines and JavaScript runtimes. High-frequency trading systems use C++ to minimize latency. Database engines, machine learning frameworks (TensorFlow, PyTorch), computer-aided design software, and aerospace control systems all rely heavily on C++.

The language is also used in large-scale desktop and server applications. Adobe's creative suite, Microsoft Office, and the core components of many operating systems are implemented in C++. The combination of performance, abstraction, and portability makes it suitable for applications that must run efficiently across diverse hardware platforms.

1.5 Strengths and Weaknesses

C++ offers several distinct strengths. Performance is the primary one: well-written C++ code can approach the speed of hand-tuned assembly while remaining portable. The language gives the programmer fine-grained control over memory layout, allocation strategy, and instruction sequencing. Zero-overhead abstractions mean that using classes, templates, and higher-order functions does not incur runtime penalties beyond what a hand-rolled C equivalent would cost. Deterministic object lifetimes, governed by scope, enable the RAII (Resource Acquisition Is Initialization) idiom, which ties resource management to object construction and destruction, eliminating whole classes of bugs related to resource leaks.

The language's multi-paradigm nature allows developers to choose the right approach for each problem. Procedural code for straightforward algorithms, object-oriented design for modeling complex stateful entities, generic programming for reusable containers and algorithms, functional programming for concurrent and parallel operations: all coexist and interoperate.

C++ also has well-known weaknesses. Its complexity is legendary. The language specification runs to thousands of pages, and the interactions between features can be subtle

and surprising. Undefined behavior lurks in many common operations, and the compiler offers no guarantee of catching these errors. Build times for large projects can be slow, though modules in C++20 address this partially. The standard library, while powerful, has an inconsistent design because of its evolutionary history. Memory safety remains a major concern; while modern idioms and smart pointers greatly reduce the risk, the language does not prevent raw pointer errors by default. Writing correct multithreaded code in C++ requires deep knowledge of the memory model and synchronization primitives.

1.6 The C++ Standardization Process

C++ is governed by the ISO/IEC 14882 standard, maintained by the JTC1/SC22/WG21 working group. The committee meets several times per year to review proposals, discuss defects, and plan future directions. Major releases occur roughly every three years. Each release introduces new features, deprecates outdated constructs, and refines the language and library. Compiler vendors (GCC, Clang, MSVC) implement the new standards over subsequent releases.

Understanding the standard version you are targeting is important because many features are only available from a certain standard onward. In this book, we assume a baseline of C++17, with C++20 and C++23 features noted explicitly. The default compilation mode in most compilers is C++14 or C++17; you must specify the standard flag (e.g., `-std=c++20`) to enable newer features.

1.7 The Compilation Process in Detail

A C++ program begins as one or more source files, typically with the `.cpp` extension, along with header files, typically `.hpp` or `.h`. The compilation process transforms these text files into an executable binary through several distinct stages.

1.7.1 Preprocessing

The preprocessor handles directives that begin with `#`. It performs file inclusion (`#include`), macro expansion (`#define`), conditional compilation (`#ifdef`, `#ifndef`, `#if`), and a few other tasks. The output of preprocessing is a single translation unit: the original source file with all included headers recursively expanded, all macros replaced, and conditional sections removed. This translation unit is a pure C++ source without any preprocessor directives.

Header files conventionally contain declarations: function prototypes, class definitions, constant definitions, template implementations, and inline function definitions. They are the interface through which translation units communicate with one another. A source file includes the headers it needs, and the preprocessor stitches them together into the translation unit seen by the compiler proper.

1.7.2 Compilation Proper

The compiler translates the preprocessed translation unit into an object file (typically `.o` or `.obj`). This process involves lexical analysis (breaking the text into tokens), parsing (building an abstract syntax tree), semantic analysis (type checking, overload resolution, template instantiation), optimization, and code generation. The object file contains machine code for the functions defined in the translation unit, along with data for global and static variables, and a symbol table listing the symbols defined and referenced.

At this stage, function calls to code in other translation units are left as unresolved references. The compiler does not need to see the bodies of those functions; it only needs their declarations to verify correct usage.

1.7.3 Linking

The linker combines multiple object files and libraries into a single executable or shared library. It resolves cross-references: if object file A calls a function defined in object file B, the linker patches the call instruction in A to point to the address of the function in B. The linker also performs whole-program optimizations in some build configurations (link-time optimization, LTO).

The distinction between compilation and linking is fundamental to understanding C++ build errors. "Undefined reference" errors come from the linker, indicating that a declaration was seen but no definition was provided in any linked translation unit. "Multiple definition" errors also come from the linker, indicating that the same symbol was defined in more than one object file.

1.8 Setting Up Your Environment

To write and run C++ programs, you need a compiler toolchain. The three major compilers are GCC (the GNU Compiler Collection), Clang (the LLVM compiler), and MSVC (Microsoft Visual C++). GCC and Clang are available on Linux and macOS; on Windows, you can use MinGW-w64 (a port of GCC), Clang, or MSVC.

For beginners, an Integrated Development Environment (IDE) simplifies the process of editing, compiling, and debugging. Visual Studio (on Windows), CLion (cross-platform), and Qt Creator are popular choices. VS Code with the C++ extension is a lighter-weight option that works well across platforms.

Build systems automate the process of compiling multiple source files, managing dependencies, and linking. CMake is the de facto standard for cross-platform C++ projects. It generates platform-specific build files (Makefiles on Linux, Visual Studio solutions on Windows, Xcode projects on macOS). A minimal `CMakeLists.txt` file might look like:

```
cmake_minimum_required(VERSION 3.16)
project(MyProject LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
add_executable(my_program main.cpp helper.cpp)
```

1.9 Your First Program

```
#include <iostream>

int main() {
    cout << "Hello, world.\n";
    return 0;
}
```

Save this to a file named `main.cpp`. Compile it with GCC: `g++ -std=c++20 -o hello main.cpp`. Run it with `./hello`. The program prints "Hello, world." and exits. The `#include <iostream>` directive brings in the standard input/output stream declarations. `int main()` is the entry point function; every C++ program must have exactly one `main` function with

return type `int`. `cout` is the standard output stream, and `<<` is the stream insertion operator. `\n` is a newline character. The `return 0;` indicates successful completion to the operating system; it is optional in `main` in C++ and later standards, where falling off the end of `main` implicitly returns 0.

Chapter 2

Basic Syntax and Program Structure

2.1 Lexical Elements

A C++ program is a sequence of tokens. The compiler's lexer groups characters into tokens: keywords, identifiers, literals, operators, and punctuation. Whitespace (spaces, tabs, newlines) separates tokens but is otherwise ignored except inside string literals and preprocessor directives.

Keywords are reserved words like `int`, `return`, `if`, `class`. They cannot be used as identifiers. Identifiers are names chosen by the programmer for variables, functions, classes, and other entities. They consist of letters, digits, and underscores, must not begin with a digit, and are case-sensitive. Convention in C++ favors snake_case for variables and functions, CamelCase for class names, and ALL_CAPS for macros.

Literals represent fixed values directly in code. Integer literals (`42`), floating-point literals (`3.14`), character literals (`'a'`), string literals (`"hello"`), boolean literals (`true`, `false`), and the pointer literal (`nullptr`) are the most common. Integer literals can be specified in decimal, octal (prefix `0`), hexadecimal (prefix `0x`), or binary (prefix `0b`). Suffixes can specify the type: `42u` is an unsigned int, `42L` is a long, `42ull` is an unsigned long long.

Comments are ignored by the compiler. Single-line comments begin with `//` and extend to the end of the line. Multi-line comments are enclosed between `/*` and `*/`. They do not nest. Use comments to explain why the code does something, not what it does; the code itself should make the "what" clear through good naming and structure.

2.2 Statements and Expressions

A statement is a complete unit of execution. Simple statements end with a semicolon. Compound statements, also called blocks, are sequences of statements enclosed in braces `{...}`. A block creates a new scope; variables declared within it are destroyed when execution leaves the block.

An expression is a combination of operators and operands that computes a value. An expression statement is an expression followed by a semicolon: `x = 5;` is an expression statement where the expression is an assignment. Not all expressions are statements; for example, `x + y` is an expression, but it must be part of a larger construct (such as an assignment or return statement) to be syntactically complete.

The null statement is a lone semicolon `;`. It does nothing and is occasionally used where the language requires a statement but no action is needed.

2.3 The main Function

Every C++ program begins execution in a function called `main`. It has two standard forms:

```
int main() { /* ... */ }
int main(int argc, char* argv[]) { /* ... */ }
```

The second form receives command-line arguments. `argc` is the number of arguments, and `argv` is an array of C-strings containing the arguments, with `argv[0]` being the program's name. The return value of `main` is passed to the operating system as the exit code;

0 means success, nonzero means failure. If no return statement is executed, the effect is `return 0;`.

2.4 Scope and Lifetime

The scope of a name is the region of program text in which that name is visible and can be used. C++ has several overlapping scope kinds.

Block scope applies to names declared within a function or a compound statement. They are visible from the point of declaration to the end of the enclosing block.

Namespace scope applies to names declared outside any function or class, possibly within a namespace. They are visible from the declaration point to the end of the translation unit, and can be accessed from other translation units through linkage.

Class scope applies to names declared within a class definition. They are accessible using the dot or arrow operators on objects of that class, or directly within member functions of the class.

The lifetime of an object is the period during which it exists in memory during program execution. Objects with automatic storage duration (local variables not declared `static`) are created when the block is entered and destroyed when the block is exited. Objects with static storage duration (global variables, static locals, static class members) are created before `main` begins and destroyed after `main` returns. Objects with dynamic storage duration are explicitly created with `new` and destroyed with `delete`. Objects with thread-local storage duration are created when a thread starts and destroyed when it ends.

Understanding scope and lifetime is essential for avoiding dangling references, use-after-free bugs, and resource leaks. RAII leverages automatic storage duration to manage resources by tying resource acquisition to object construction and release to destruction.

Chapter 3

Variables, Types, and Initialization

3.1 The Type System

C++ is a statically typed language: the type of every variable and expression is known at compile time. The type determines the size of the object in memory, the set of values it can hold, and the operations that can be performed on it. Strong static typing enables the compiler to catch many errors early and to generate efficient machine code.

Types in C++ are divided into fundamental types and compound types. Fundamental types include arithmetic types (integral and floating-point), `void`, and `nullptr_t`. Compound types include pointers, arrays, references, functions, enumerations, and classes.

3.2 Fundamental Types

The integral types represent whole numbers. Their sizes are platform-dependent, but the standard guarantees minimum ranges. `char` is at least 8 bits, and it is a distinct type from both `signed char` and `unsigned char`, which are also 8-bit integral types. `short` is at least 16 bits, `int` at least 16 bits (in practice 32 on modern systems), `long` at least 32 bits, `long long` at least 64 bits (since C++11). Each integer type has a signed and unsigned variant, except `char8_t`, `char16_t`, and `char32_t` introduced for UTF character support.

The floating-point types are `float` (single-precision, typically 32-bit IEEE 754), `double` (double-precision, typically 64-bit), and `long double` (extended precision, platform-dependent). Floating-point arithmetic is not exact; it approximates real numbers with a finite binary representation. This leads to rounding errors, loss of precision, and surprises like `0.1 + 0.2 != 0.3`.

The Boolean type `bool` holds the values `true` and `false`. It occupies at least one byte. Conversions from integral types treat zero as `false` and anything else as `true`.

`void` is an incomplete type that cannot be completed. It is used as the return type of functions that do not return a value, and as `void*` for pointers to an object of unknown type.

`nullptr_t` is the type of the null pointer literal `nullptr`. It can be implicitly converted to any pointer type.

3.3 Variable Declarations and Definitions

A declaration introduces a name and its type into a scope. A definition also causes storage to be allocated (for variables) or code to be generated (for functions). A variable declaration that is not a definition uses `extern`:

```
extern int x;       // declaration only
int y;              // definition
```

Variables should be declared as close to their first use as possible, and preferably initialized at the point of declaration.

3.4 Initialization

Initialization is the process of giving an object an initial value when it is created. It is distinct from assignment, which overwrites an existing value. C++ has several initialization syntaxes, and their semantics have evolved across standards.

3.4.1 Default Initialization

If no initializer is provided, default initialization occurs. For built-in types at block scope, this means the variable holds an indeterminate value; reading it before assignment is undefined behavior. For class types, the default constructor is called. For built-in types at namespace scope, zero-initialization occurs, which is well-defined.

3.4.2 Value Initialization

Using empty parentheses or braces with no arguments performs value initialization. For built-in types, this zero-initializes. For class types, it calls the default constructor, or zero-initializes if the default constructor is compiler-generated.

```
int x{};   // value initialization, x is 0
int y();   // WARNING: declares a function named y returning int
```

The `int y();` pitfall is known as the "most vexing parse." It declares a function, not a variable. Use braces to avoid this.

3.4.3 Direct Initialization

Direct initialization uses parentheses to invoke a constructor or to initialize a built-in type:

```
int x(5);
string s("hello");
```

3.4.4 Copy Initialization

Copy initialization uses the equals sign:

```
int x = 5;
string s = "hello";
```

For class types, this may implicitly call a converting constructor and then the copy constructor, though the compiler typically elides the copy.

3.4.5 List Initialization and Uniform Initialization

Since C++11, brace initialization (list initialization) provides a uniform syntax that works everywhere and prevents narrowing conversions:

```
int x{5};
vector<int> v{1, 2, 3};
int arr[]{1, 2, 3};
```

Narrowing conversions (converting a value to a type that cannot represent it, such as `int` to `char` with possible data loss) are prohibited in brace initialization and cause a compilation error. This is a significant safety improvement.

3.4.6 Aggregate Initialization

Aggregates (arrays and classes with no user-provided constructors, no private or protected non-static data members, no virtual functions) can be initialized with brace-enclosed initializer lists. Since C++20, aggregate initialization supports designated initializers:

```
struct Point { int x; int y; };
Point p{ .x = 10, .y = 20 };
```

3.5 Type Deduction

3.5.1 auto

The `auto` keyword deduces the type of a variable from its initializer. It strips references and top-level const:

```
auto x = 5;          // int
auto y = 5.0;        // double
const int& z = x;
auto w = z;          // int, not const int&
auto& wr = z;        // const int&
```

Use `auto` when the type is obvious from the initializer or when the exact type is cumbersome to write (e.g., iterator types). Avoid it when the type is not clear from context, as it can harm readability.

3.5.2 decltype

`decltype` gives the declared type of an expression without evaluating it:

```
int x = 5;
decltype(x) y = 10;           // int
decltype((x)) z = x;          // int& (parentheses make it an lvalue
    expression)
```

`decltype` is essential in generic programming when the type of an expression depends on template parameters.

3.6 const, constexpr, consteval, constinit

`const` declares that a variable's value cannot be changed after initialization:

```
const int max_size = 100;
```

A `const` object can be initialized with a runtime value. It is a promise not to modify, not necessarily a compile-time constant.

`constexpr`, since C++11, indicates that a variable or function can be evaluated at compile time. A `constexpr` variable must be initialized with a constant expression. A `constexpr` function can be called at compile time or at runtime; if all its arguments are constant expressions, the call is evaluated at compile time:

```
constexpr int square(int x) { return x * x; }
constexpr int val = square(10);  // computed at compile time
```

`consteval` (C++20) creates an immediate function that must be evaluated at compile time. Calling it with non-constant arguments is an error:

```
consteval int cube(int x) { return x * x * x; }
// cube(runtime_value);  // error
```

`constinit` (C++20) ensures that a variable with static storage duration is initialized at compile time. Unlike `constexpr`, the variable itself is not const:

```
constinit int global_counter = 0;  // must be constant-initialized
```

3.7 Type Modifiers and Qualifiers

`const` and `volatile` are cv-qualifiers. `volatile` tells the compiler that a variable's value may change outside the program's control (e.g., memory-mapped hardware registers). It disables optimizations that assume the value does not change between reads. In modern C++, `volatile` is rarely used; it is not sufficient for multithreading (use atomics instead).

`mutable` allows a data member to be modified even on a const object. It is typically used for caching, mutexes, or other implementation details that do not affect the observable state of the object.

`static` has multiple meanings. At namespace scope, it gives internal linkage (the name is local to the translation unit). In a class, a static data member is shared by all objects of the class. In a function, a static local variable is initialized once and retains its value across calls.

`extern` declares a variable or function defined in another translation unit.

3.8 Type Aliases

Type aliases give alternative names to types. The traditional syntax uses `typedef`; the modern syntax uses `using`:

```
typedef unsigned long ulong;   // traditional
using ulong = unsigned long;   // modern, preferred
```

The `using` syntax is more readable for complex types and is required for alias templates.

Chapter 4

Operators and Expressions

4.1 Overview

C++ provides a rich set of operators for arithmetic, comparison, logic, bit manipulation, and memory access. Understanding operator precedence and associativity is critical for reading and writing correct code. When in doubt, use parentheses to make intent explicit; do not rely on the reader's memory of precedence tables.

4.2 Arithmetic Operators

The binary arithmetic operators are `+`, `-`, `*`, `/`, and `%` (modulo). Integer division truncates toward zero. Modulo with negative operands has implementation-defined sign in C++98 but is well-defined since C++11: `(a/b)*b + a%b == a`, and the sign of the remainder equals the sign of the dividend. The unary `+` and `-` operators are also available.

Beware of integer overflow. For signed integers, overflow is undefined behavior. For unsigned integers, overflow wraps around modulo 2^N where N is the number of bits. This difference can cause bugs in security-critical code and makes signed overflow a serious concern. Use compiler flags like `-ftrapv` or sanitizers to detect overflow.

Floating-point arithmetic can produce infinities and NaN (Not a Number) values. Division by zero in floating-point produces infinity, not a crash. Operations with NaN produce NaN. Comparing with NaN always yields false, even `NaN == NaN` is false. Use `isnan()` to test for NaN.

4.3 Comparison and Relational Operators

The comparison operators are `==`, `!=`, `<`, `>`, `<=`, `>=`. They return `bool`. For fundamental types, these work as expected. For pointers, comparing pointers to different objects (except for certain cases like arrays) is unspecified. Use `less` and friends for total ordering of pointers.

Since C++20, the three-way comparison operator `<=>` (the "spaceship operator") is available. It returns an object of a comparison category type (`strong_ordering`, `weak_ordering`, or `partial_ordering`) that can be compared to 0. Defining `<=>` for a class can automatically generate all six comparison operators.

4.4 Logical Operators

`&&` (and), `||` (or), and `!` (not) operate on boolean values. They perform short-circuit evaluation: the right operand of `&&` is evaluated only if the left is true; the right operand of `||` is evaluated only if the left is false. This is not merely an optimization but a semantic guarantee used for conditional execution:

```
if (ptr != nullptr && ptr->value > 0) { /* safe dereference */ }
```

The alternative tokens `and`, `or`, `not` are also available but rarely used in modern codebases.

4.5 Bitwise Operators

`&`, `|`, `^`, `~`, `«`, `»` manipulate individual bits of integral operands. Left shift (`«`) shifts bits to the left, filling with zeros. Right shift (`»`) with unsigned types fills with zeros (logical shift). Right shift with signed negative values is implementation-defined; in practice it is usually an arithmetic shift (fills with the sign bit). Bitwise operations are fundamental to embedded programming, protocol parsing, and performance-critical flag manipulation.

Do not confuse bitwise `&` and `|` with logical `&&` and `||`. They have different precedence and semantics.

4.6 Assignment Operators

The simple assignment `=` stores a value in a variable. Compound assignments (`+=`, `-=`, `*=`, `/=`, `%=`, `«=`, `»=`, `&=`, `|=`, `≙`) apply the operation and then assign. Assignment is an expression, and its value is a reference to the left-hand side. This allows chaining: `a = b = c = 0;`. Assignments associate right-to-left.

4.7 Increment and Decrement

The prefix forms (`++x`, `-x`) modify the variable and return a reference to the new value. The postfix forms (`x++`, `x-`) return a copy of the old value, then modify the variable. For built-in types, the difference is purely about the value of the expression. For iterators and user-defined types, postfix can be more expensive because it must make a copy. Prefer prefix forms unless you specifically need the old value.

4.8 The Conditional Operator

The ternary operator `?:` selects between two values based on a condition:

```
int max = (a > b) ? a : b;
```

The condition is evaluated first. If true, the expression after `?` is evaluated and the other is skipped; if false, the reverse. Both branches must have compatible types. Use the conditional operator for simple selections; avoid nesting it deeply.

4.9 The Comma Operator

The comma operator evaluates its left operand, discards the result, then evaluates the right operand and yields its value. It is most often seen in `for` loop headers but is otherwise rarely useful and can be confusing. Avoid it outside of well-established patterns.

4.10 Member Access Operators

The dot operator `obj.member` accesses members of an object. The arrow operator `ptr->member` is syntactic sugar for `(*ptr).member`. The `.*` and `->*` operators work with pointers to members, an advanced feature.

4.11 Address-of and Dereference

`&x` yields the address of variable `x`, producing a pointer. `*ptr` dereferences a pointer, yielding a reference to the pointed-to object. These are fundamental to pointer-based programming but should be used sparingly in modern C++ where references and smart pointers are preferred.

4.12 sizeof and alignof

`sizeof` returns the size in bytes of a type or expression. `alignof` (C++11) returns the alignment requirement. These are evaluated at compile time for complete types. Use them when writing allocators, serialization code, or low-level data structure implementations.

4.13 Operator Precedence and Associativity

Precedence determines which operators bind tighter in expressions without parentheses. Associativity determines the grouping direction for operators of the same precedence. The following groups are listed from highest to lowest precedence:

- Scope resolution: `::`
- Postfix: `()`, `[]`, `->`, `.`, `++`, `--`
- Unary prefix: `++`, `--`, `*`, `&`, `+`, `-`, `!`, `~`, `sizeof`, `alignof`
- Member pointer: `.*`, `->*`
- Multiplicative: `*`, `/`, `%`
- Additive: `+`, `-`
- Shift: `<<`, `>>`
- Three-way comparison: `<=>`
- Relational: `<`, `>`, `<=`, `>=`
- Equality: `==`, `!=`
- Bitwise AND: `&`
- Bitwise XOR: `^`
- Bitwise OR: `|`
- Logical AND: `&&`
- Logical OR: `||`
- Conditional: `?:`
- Assignment: `=`, `+=`, etc.
- Comma: `,`

When reading or writing complex expressions, use parentheses liberally. The compiler does not charge for them, and they prevent bugs.

Chapter 5

Control Flow

5.1 Conditional Statements

The `if` statement executes a block of code if a condition is true. An optional `else` branch executes if the condition is false. The condition must be convertible to `bool`. Nesting `if` statements is straightforward, but deeply nested conditionals become hard to read. Use early returns, guard clauses, or polymorphism to reduce nesting.

Since C++17, an `if` statement can have an initializer:

```
if (auto it = map.find(key); it != map.end()) {
    use(it->second);
} else {
    // key not found
}
```

This limits the scope of the variable to the `if` and `else` blocks.

The `switch` statement dispatches on an integral or enum value. Each `case` label must be a constant expression. Execution falls through to the next case unless a `break` statement is encountered. Fallthrough is a common source of bugs; C++17 added the `[[fallthrough]];` attribute to signal intentional fallthrough:

```
switch (command) {
    case CMD_START:
        start();
        [[fallthrough]];
    case CMD_RESUME:
        resume();
        break;
    case CMD_STOP:
        stop();
        break;
    default:
        handle_unknown();
}
```

Always include a `default` case, even if it only contains a comment, to make the programmer's intent explicit and to suppress compiler warnings.

5.2 Looping Constructs

The `while` loop executes its body as long as the condition is true. The condition is evaluated before each iteration, so the body may execute zero times.

```
while (condition) {
    // body
}
```

The `do-while` loop executes its body at least once, then evaluates the condition:

```
do {
```

```
    // body
} while (condition);
```

Use `do-while` when the body must execute before the condition can be meaningfully evaluated, such as reading user input.

The `for` loop has three clauses: initialization, condition, and increment:

```
for (int i = 0; i < n; ++i) {
    // body
}
```

All three clauses are optional. The traditional C-style `for` loop is powerful but error-prone. Since C++11, the range-based `for` loop iterates over any range with `begin()` and `end()`:

```
for (const auto& element : container) {
    // read-only access to element
}
```

Use range-based `for` whenever possible; it eliminates off-by-one errors and iterator invalidation concerns.

5.3 Jump Statements

`break` exits the innermost enclosing loop or `switch`. `continue` skips the remainder of the current loop iteration and proceeds to the next condition evaluation. `goto` unconditionally transfers control to a labeled statement in the same function. The label syntax is `label_name:`. `goto` is rarely appropriate; its legitimate uses include breaking out of deeply nested loops and cleanup in error handling where RAII cannot be used.

`return` exits the current function and optionally provides a value. In a non-void function, every control path must return a value (or throw an exception). Falling off the end of a non-void function is undefined behavior except for `main`.

5.4 Control Flow and Object Lifetime

When control exits a block, either normally or via `break`, `continue`, `goto`, or `return`, all automatic objects declared in that block are destroyed in reverse order of construction. This is the foundation of RAII. For example, a lock guard object acquiring a mutex in its constructor will release it in its destructor when the block exits, regardless of how the exit occurs:

```
{
    lock_guard<mutex> lock(my_mutex);
    // critical section
} // lock released here, even if an exception is thrown
```

Chapter 6

Functions

6.1 Function Basics

A function encapsulates a reusable block of code. It has a name, a return type, a parameter list, and a body. The function declaration (prototype) states the name and types; the definition provides the implementation. A function must be declared before use; the definition can be in a separate translation unit.

```
int add(int a, int b);  // declaration
int add(int a, int b) { return a + b; }  // definition
```

Parameters are local variables initialized with the arguments passed at the call site. Arguments are passed by value by default: the function receives a copy. Changes to the parameter inside the function do not affect the caller's argument.

6.2 Pass by Reference

To allow a function to modify the caller's argument, or to avoid copying large objects, pass by reference. A reference parameter is an alias for the argument:

```
void increment(int& x) { ++x; }
```

To pass a large object without copying but also without permitting modification, use a const reference:

```
void print(const string& s) { cout << s; }
```

Passing by const reference is the default choice for non-primitive types in C++. It is efficient and expresses the intent of read-only access.

6.3 Default Arguments

Parameters can have default values specified in the declaration (or definition if no separate declaration exists). Default arguments must be rightmost:

```
void print_log(const string& msg, int level = 0);
print_log("error");       // level defaults to 0
print_log("critical", 3);
```

Default arguments are evaluated at the call site, not at the declaration site, which matters for expressions with side effects or runtime context.

6.4 Function Overloading

Multiple functions can have the same name if their parameter lists differ in number or types. The return type alone is not sufficient to distinguish overloads. The compiler resolves the call by matching arguments to parameters using a complex set of rules (overload resolution). Overloaded functions should perform semantically similar operations; do not use overloading for unrelated behaviors.

Ambiguities arise when implicit conversions make multiple overloads equally viable. For example, calling `f(5)` where `f(int)` and `f(long)` exist is ambiguous because `int` to `long` is a conversion of equal rank as calling `f(5)` directly. Use explicit casts or adjust the overload set to resolve ambiguities.

6.5 Inline Functions

The `inline` keyword suggests that the compiler should replace calls to the function with the function's body, avoiding call overhead. Compilers are free to ignore this hint and make their own inlining decisions based on optimization heuristics. In modern C++, `inline` primarily affects linkage: an inline function may be defined in multiple translation units (e.g., in a header) without violating the one-definition rule.

Functions defined entirely within a class definition are implicitly inline. Use `inline` for small functions defined in headers; for larger functions, rely on compiler optimization and link-time optimization.

6.6 Recursion

A function that calls itself is recursive. Recursion is natural for problems with self-similar structure: tree traversals, divide-and-conquer algorithms, parsing. Each recursive call adds a new stack frame; if the recursion is too deep, the stack overflows. Always provide a base case that terminates the recursion. Tail recursion (where the recursive call is the last action) can be optimized by the compiler to reuse the stack frame, effectively turning recursion into iteration, but this optimization is not guaranteed in C++.

6.7 Function Pointers

A function pointer stores the address of a function. It can be used to pass behavior as an argument:

```
using Comparator = bool(*)(int, int);
void sort(int* data, size_t size, Comparator comp);

bool ascending(int a, int b) { return a < b; }
sort(arr, n, ascending);
```

Function pointers are a simple form of callable object, but lambdas and `function` are more flexible.

6.8 Lambda Expressions

A lambda expression creates an anonymous function object (a closure). Since C++11, lambdas are a core feature for functional and generic programming:

```
auto less = [](int a, int b) { return a < b; };
```

The brackets are the capture clause, specifying which variables from the enclosing scope the lambda can access. Capture by value `[=]` copies variables; capture by reference `[&]` allows modification of originals. Specific variables can be captured: `[x, &y]`. The capture clause is followed by the parameter list, the mutable keyword if needed (for modifying by-value captures), the return type if deduced return is insufficient, and the body.

Lambdas are syntactic sugar for creating a class with `operator()`. They can be stored in variables, passed to algorithms, and used as callbacks. Generic lambdas (C++14) use `auto` in parameter types:

```
auto generic = [](auto x, auto y) { return x + y; };
```

Stateless lambdas (with empty capture) are implicitly convertible to function pointers.

6.9 function

`function` is a polymorphic function wrapper that can hold any callable object: function pointers, lambdas, function objects. It has a fixed signature and some runtime overhead due to type erasure:

```
function<int(int, int)> op = [](int a, int b) { return a + b; };
```

Prefer lambdas and templates for compile-time polymorphism; use `function` when you need to store callables of varying types in a homogeneous container or when the callable type cannot be known at compile time.

6.10 Callable Objects

Any object that defines `operator()` can be called like a function. These function objects (functors) are central to generic programming:

```
struct Multiplier {
    int factor;
    Multiplier(int f) : factor(f) {}
    int operator()(int x) const { return x * factor; }
};
Multiplier times5(5);
int result = times5(10);  // 50
```

Functors have advantages over plain functions: they can carry state, and their call can be inlined more easily because the compiler knows the concrete type at the call site.

6.11 Return Type Deduction

Since C++14, `auto` can be used as the return type, and the compiler deduces it from return statements. Since C++17, deduction works correctly even with multiple return paths, provided they all return the same type. For trailing return type syntax, use `auto` before the function name and `-> Type` after the parameter list. This is useful in templates when the return type depends on argument types:

```
template<typename T, typename U>
auto multiply(T a, U b) -> decltype(a * b) {
    return a * b;
}
```

Since C++14, the trailing return type can often be omitted because `auto` deduction handles it.

6.12 constexpr Functions

A `constexpr` function can be evaluated at compile time if all its arguments are constant expressions. Since C++14, `constexpr` functions can contain loops, branches, and multiple statements. This allows complex computations to happen during compilation, reducing runtime overhead. Since C++20, even `try-catch` blocks are allowed in `constexpr` functions (though throwing at compile time is an error). Use `constexpr` functions to shift work from runtime to compile time whenever possible.

Chapter 7

Arrays, Strings, and Containers

7.1 C-Style Arrays

A C-style array is a contiguous block of elements of the same type. Its size is fixed at compile time:

```
int arr[5] = {1, 2, 3, 4, 5};
```

The array name decays to a pointer to the first element in most contexts, which causes confusion and bugs. The size is not stored with the array; passing an array to a function typically requires also passing the size, and there is no bounds checking. Prefer `array` or `vector` over C-style arrays in modern code.

7.2 array

`array<T, N>` is a fixed-size array that does not decay to a pointer. It provides `size()`, bounds-checked access with `at()`, iterators, and full value semantics (copy, assign). It has no space overhead over a C-style array:

```
array<int, 5> arr = {1, 2, 3, 4, 5};
for (const auto& elem : arr) {
    cout << elem << ' ';
}
```

7.3 vector

`vector<T>` is a dynamically resizable array. It stores elements contiguously, supports random access, and manages its own memory. When capacity is exceeded, it allocates a larger block, copies or moves elements, and deallocates the old block. Amortized complexity of `push_back` is constant.

```
vector<int> vec;
vec.push_back(10);
vec.emplace_back(20);  // constructs in-place
```

Common member functions include `size()`, `capacity()`, `reserve()`, `resize()`, `clear()`, and `shrink_to_fit()`. Be aware of iterator invalidation: operations that reallocate (like `push_back` when size equals capacity) invalidate all iterators and references.

7.4 string

`string` is a specialization of `basic_string<char>`. It represents a mutable, contiguous sequence of characters. It provides concatenation, substring, search, comparison, and many other operations:

```
string s = "hello";
s += " world";
```

```
size_t pos = s.find("world");
string sub = s.substr(0, 5);
```

Since C++17, `string_view` provides a non-owning view into a character sequence. It avoids copies and is useful for function parameters that read a string without taking ownership:

```
void process(string_view sv) {
    // use sv without copying
}
```

Be careful with `string_view` lifetime; it does not extend the lifetime of the underlying data. A `string_view` into a temporary string is a dangling reference.

7.5 Iterators

Iterators abstract the traversal of containers. They provide a common interface that decouples algorithms from container implementations. Every standard container provides `begin()` and `end()` members that return iterators to the first element and one past the last. Iterator categories are input, output, forward, bidirectional, and random-access, each providing different sets of operations. All iterators support `++` to advance; random-access iterators support `+=`, `[]`, and comparisons with `<`, `>`.

The following pattern works for any container:

```
for (auto it = container.begin(); it != container.end(); ++it) {
    auto& element = *it;
    // use element
}
```

Range-based `for` loops are syntactic sugar over this pattern.

Chapter 8

Pointers and References

8.1 Pointer Fundamentals

A pointer is a variable that holds a memory address. The type `int*` is a pointer to an `int`. The unary `&` operator takes the address of an object, producing a pointer to its type. The unary `*` operator dereferences a pointer, yielding a reference to the pointed-to object:

```
int x = 42;
int* ptr = &x;
*ptr = 10;   // modifies x through the pointer
```

Pointers can be reassigned to point to different objects or to `nullptr`, which indicates that they point to nothing. Dereferencing a null pointer is undefined behavior and typically causes a segmentation fault. Always check for null before dereferencing when the pointer's validity is uncertain.

8.2 Pointer Arithmetic

Pointer arithmetic operates on the array element scale, not in raw bytes. Adding 1 to a pointer moves it to the next element, not the next byte. Subtracting two pointers yields the number of elements between them. Pointer arithmetic is only valid within the bounds of an array object or one element past the end. Accessing memory outside these bounds is undefined behavior.

C++ provides `ptrdiff_t` for differences and `size_t` for sizes. Use these types instead of raw `int` or `long`.

8.3 References

A reference is an alias for an existing object. Once initialized, a reference cannot be reseated to refer to a different object. References are not objects themselves; they do not occupy storage (though the compiler may implement them using pointers under the hood). References must be initialized at declaration.

Passing by reference is the idiomatic way to allow functions to modify arguments and to avoid copies for large objects. A const reference can bind to a temporary, extending its lifetime:

```
const string& ref = get_string();  // temporary lives as long as ref
```

8.4 Lvalue and Rvalue References

Understanding value categories is essential for mastering move semantics. An lvalue represents an object with a stable identity and location (a named variable, a dereferenced pointer). An rvalue is a temporary object, such as the result of an arithmetic expression or a function returning by value.

An lvalue reference `T&` binds to lvalues. An rvalue reference `T&&` binds to rvalues. Rvalue references were introduced in C++11 to enable move semantics. When an object is an rvalue, it is about to be destroyed, and its resources can be safely pilfered:

```
string s1 = "hello";
string s2 = move(s1);  // s2 steals s1's allocation; s1 is empty
```

`move` is a cast to rvalue reference; it does not actually move anything. The move constructor or move assignment operator of the target type performs the resource transfer.

Universal references (also called forwarding references) appear in template context as `T&&` where `T` is a deduced template parameter. They can bind to both lvalues and rvalues, and their actual type depends on the argument category. Use `forward` to perfectly forward the argument preserving its value category.

8.5 Smart Pointers

Raw pointers are error-prone: they do not convey ownership, and manual `delete` is easily forgotten or double-executed. C++11 introduced smart pointer classes that manage ownership automatically.

`unique_ptr<T>` owns the pointed-to object exclusively. It cannot be copied, only moved. When the `unique_ptr` is destroyed, it deletes the owned object. Use `make_unique` to create one:

```
auto ptr = make_unique<Widget>(args);
```

`shared_ptr<T>` shares ownership via reference counting. When the last `shared_ptr` pointing to an object is destroyed, the object is deleted. It has overhead for the control block and reference count operations. Use `make_shared` for efficient single-allocation creation.

`weak_ptr<T>` is a non-owning observer of an object managed by `shared_ptr`. It can be converted to a `shared_ptr` via `lock()` to temporarily share ownership, but it does not keep the object alive. Use `weak_ptr` to break cycles in shared ownership graphs.

8.6 Dynamic Memory Allocation

Use `new` to allocate objects on the heap (free store) and `delete` to deallocate them. In modern C++, raw `new` and `delete` should appear only in low-level resource management code; prefer smart pointers and containers.

```
int* p = new int(42);
// ...
delete p;
```

Array `new[]` must be paired with `delete[]`. Mismatched scalar and array forms are undefined behavior. Use `vector` instead of `new[]` to avoid this entirely.

Placement `new` constructs an object in already-allocated memory. It is used in allocators, object pools, and when custom memory management is needed. Placement `new` does not allocate; it only calls the constructor. No `delete` is called; the destructor must be invoked explicitly.

Chapter 9

Structures and Classes

9.1 From struct to class

C++ inherits the `struct` keyword from C, but in C++ a struct can have member functions, constructors, destructors, and access specifiers. The only difference between `struct` and `class` is the default access: `struct` members are public by default; `class` members are private. Conventionally, `struct` is used for simple data aggregates, and `class` for types with invariants and private implementation.

9.2 Member Functions

A member function is a function declared within a class. It has an implicit `this` pointer referring to the object on which it was called. Member functions can be defined inside the class (implicitly inline) or outside using the scope resolution operator:

```
class Counter {
    int count_ = 0;
public:
    void increment() { ++count_; }
    int value() const { return count_; }
};
```

The `const` after the parameter list declares that the member function does not modify the object; it can be called on const objects. Const correctness is a foundational practice: mark every member function that does not modify the object as `const`.

9.3 Constructors and Destructors

A constructor is a special member function that initializes an object. It has the same name as the class and no return type. If no constructor is declared, the compiler generates a default constructor that default-initializes members. Once any constructor is defined, the generated default constructor is suppressed.

The destructor, named `~ClassName`, is called when the object's lifetime ends. It should release resources owned by the object. The compiler-generated destructor destroys members in reverse declaration order.

Constructor initializer lists initialize members before the constructor body executes. Use initializer lists instead of assignment in the body for efficiency and to initialize const and reference members:

```
class Person {
    string name_;
    int age_;
public:
    Person(const string& name, int age)
        : name_(name), age_(age) {}
};
```

9.4 Copy Semantics

When an object is copied, the compiler copies each member. For built-in types, this is a bitwise copy. For pointer members, this copies the pointer, not the pointed-to resource (a shallow copy). Shallow copies lead to double deletion and dangling pointers when two objects think they own the same resource.

The copy constructor `Class(const Class&)` and copy assignment operator `Class& operator=(const Class&)` define copying behavior. The compiler generates them automatically, performing memberwise copy. If your class manages a resource (file handle, heap memory, lock), you must define these operations (or delete them) to handle the resource correctly.

9.5 Move Semantics

Move semantics allow the transfer of resources from a source object to a destination, leaving the source in a valid but unspecified state. The move constructor `Class(Class&&)` and move assignment operator `Class& operator=(Class&&)` are called when the source is an rvalue. Move operations are typically noexcept and should leave the source object in a state where it can be safely destroyed or assigned to.

The compiler generates move operations automatically only if no copy operations, move operations, or destructor are user-declared. This behavior underscores the Rule of Three and Rule of Five.

9.6 Rule of Three, Five, and Zero

The Rule of Three states: if a class needs a user-defined destructor, copy constructor, or copy assignment operator, it almost certainly needs all three. This is because each of these functions indicates non-trivial resource management.

The Rule of Five extends this to move operations: if a class needs any of the five special member functions (destructor, copy constructor, copy assignment, move constructor, move assignment), it probably needs all five.

The Rule of Zero is the modern recommendation: classes that do not manage resources should not declare any special member functions. Use resource-managing types (smart pointers, containers) as members, and the compiler-generated defaults will be correct.

9.7 Inheritance

Inheritance models an "is-a" relationship. A derived class inherits members and behaviors from a base class. Public inheritance means the derived class is-a base class and can be used polymorphically:

```
class Animal {
public:
    virtual void speak() const { cout << "?\n"; }
    virtual ~Animal() = default;
};

class Dog : public Animal {
public:
    void speak() const override { cout << "Woof\n"; }
};
```

Virtual functions enable runtime polymorphism. The `virtual` keyword in the base class declares that the function may be overridden in derived classes. The `override` keyword in the derived class ensures that the function actually overrides a base virtual function; it is a compile-time check that prevents signature mismatches.

The destructor of a polymorphic base class must be virtual. If it is not, deleting a derived object through a base pointer invokes undefined behavior. When virtual, the correct destructor is called from the most derived class upward.

9.8 Polymorphism

Polymorphism allows code written in terms of a base class interface to operate on objects of different derived types. Pointers and references to the base class can refer to derived objects. Virtual function calls are dispatched at runtime based on the actual type of the object:

```
void interact(const Animal& a) {
    a.speak();  // calls Dog::speak, Cat::speak, etc.
}
```

Pure virtual functions (`virtual void f() = 0;`) make a class abstract; it cannot be instantiated. Derived classes must override pure virtuals to become concrete. Abstract classes define interfaces.

9.9 Composition and Aggregation

Composition models a "has-a" relationship. A class contains objects of other types as members. Composition implies ownership; when the owning object is destroyed, its members are destroyed. Aggregation is a weaker relationship where the contained object exists independently and may be shared.

Favor composition over inheritance. Inheritance creates tight coupling and is appropriate only when the derived class truly substitutes for the base in all contexts. Composition and delegation are more flexible and less error-prone.

Chapter 10

Operator Overloading

Operators can be overloaded for user-defined types, allowing objects to work with standard operator syntax. Most operators can be overloaded, including arithmetic, comparison, assignment, call, subscript, and dereference operators. The following cannot be overloaded: `::`, `.*`, `?:`, and a few others.

Operator overloading should be intuitive. The overloaded operator should behave like its built-in counterpart. Do not overload operators to do something surprising or unrelated to their conventional meaning.

Operators can be overloaded as member functions or as non-member functions. Binary operators like `+` are often implemented as non-members to allow implicit conversions on both operands:

```
class Complex {
    double re, im;
public:
    Complex(double r, double i) : re(r), im(i) {}
    friend Complex operator+(const Complex& a, const Complex& b);
};
Complex operator+(const Complex& a, const Complex& b) {
    return Complex(a.re + b.re, a.im + b.im);
}
```

Assignment operators should return `*this` by reference. The function call operator `operator()` creates a functor. The subscript operator `operator[]` provides array-like access. The dereference operators `operator*`, `operator->` create smart pointer and iterator types.

The increment and decrement operators have two forms: prefix (`operator++()`) and postfix (`operator++(int)`), where the `int` parameter is a dummy to distinguish them.

Chapter 11

Namespaces

Namespaces partition the global scope into named sub-scopes, preventing name collisions. The standard library lives in the `std` namespace.

```
namespace my_project {
    class Connection { /* ... */ };
    void connect(Connection& c);
}
```

Names can be accessed with the scope resolution operator: `my_project::Connection`. The `using` directive (`using namespace std;`) pulls all names from a namespace into the current scope; it is convenient but should be avoided in header files and at global scope in implementation files, because it defeats the purpose of namespaces. Prefer `using` declarations for specific names: `using cout;`. This limits the scope of imported names.

Anonymous namespaces provide internal linkage, replacing `static` for file-local entities. Namespace aliases shorten long qualified names: `namespace fs = filesystem;`. The global namespace is the default scope for names not declared in any explicit namespace.

Chapter 12

Templates

12.1 Function Templates

A function template defines a family of functions parameterized by type:

```
template<typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}
```

The compiler deduces `T` from the argument types at the call site. If deduction fails or is ambiguous, the programmer can specify the type explicitly: `max<double>(3, 4.5)`. Template code is instantiated on demand: the compiler generates a concrete function for each unique combination of template arguments used in the program. This means template definitions must be available in every translation unit that uses them, typically by placing them in header files.

12.2 Class Templates

A class template is a blueprint for generating classes:

```
template<typename T>
class Stack {
    vector<T> elems;
public:
    void push(const T& elem) { elems.push_back(elem); }
    T pop() { /* ... */ }
};
Stack<int> intStack;
```

Member functions of class templates are themselves templates. They can be defined inside or outside the class; outside definitions require the `template` prefix and qualified name.

12.3 Template Specialization

Explicit specialization provides a custom implementation for a specific set of template arguments:

```
template<>
class Stack<bool> {
    // specialized implementation using bits
};
```

Partial specialization applies to class templates (not function templates) and allows specialization for a subset of template parameters, e.g., a generic `Stack` for pointer types.

12.4 Variadic Templates

Since C++11, templates can accept any number of template parameters using an ellipsis:

```
template<typename... Args>
void print(Args... args) {
    (cout << ... << args);  // fold expression (C++17)
}
```

Parameter packs can be expanded using unpacking syntax. Fold expressions (C++17) apply a binary operator over a parameter pack, greatly simplifying variadic template code.

12.5 Concepts (C++20)

Concepts constrain template parameters, providing clearer error messages and enabling overloading based on constraints:

```
template<typename T>
concept Numeric = is_arithmetic_v<T>;

template<Numeric T>
T add(T a, T b) { return a + b; }
```

The `requires` clause can also be used for ad-hoc constraints. Concepts make templates more robust and self-documenting. They are the preferred way to express template requirements in C++20 and later.

Chapter 13

Exception Handling

13.1 Error Handling Philosophy

C++ provides exception handling for separating error detection from error recovery. A function that encounters an unrecoverable situation throws an exception; calling code that can handle the situation catches it. Exceptions are not for normal control flow; they are for exceptional circumstances. The zero-overhead principle applies: if no exception is thrown, there should be no runtime cost.

13.2 Throwing and Catching

Any copyable or movable object can be thrown. Throwing an exception unwinds the stack, destroying all automatic objects in the intermediate scopes (stack unwinding), until a matching catch block is found. If no matching handler is found, `terminate` is called.

```
try {
    risky_operation();
} catch (const runtime_error& e) {
    cerr << "Runtime error: " << e.what() << '\n';
} catch (const exception& e) {
    cerr << "Standard exception: " << e.what() << '\n';
} catch (...) {
    cerr << "Unknown exception\n";
    throw;  // re-throw
}
```

Catch by reference to avoid slicing. Catch the most derived types first. The `catch (...)` handler catches anything; it should be used sparingly, typically to perform cleanup before re-throwing or terminating.

13.3 noexcept

The `noexcept` specifier declares that a function does not throw. It is part of the function's type since C++17. If a `noexcept` function does throw (e.g., by calling a non-noexcept function that throws), `terminate` is called. Move constructors and destructors should be noexcept; the standard library containers prefer moving elements only if the move operation is noexcept, falling back to copying otherwise.

`noexcept` can also be used as an operator: `noexcept(expr)` returns true if `expr` is noexcept. This is useful in templates to conditionally enable noexcept based on type properties.

13.4 RAII and Exceptions

RAII is the cornerstone of exception safety. Because destructors are called during stack unwinding, resources are released automatically. Classes should use the RAII idiom: acquire resources in the constructor, release in the destructor. This guarantees correctness even in the presence of exceptions.

Exception safety guarantees come in three levels: no-throw (the operation cannot fail), strong guarantee (if an exception is thrown, the state is rolled back to before the operation), and basic guarantee (no resources are leaked and invariants are preserved, but the state may be modified). Design your functions to provide the strongest guarantee possible.

Chapter 14

Move Semantics in Depth

14.1 Value Categories Revisited

C++ classifies every expression into one of three primary value categories: lvalue, xvalue, and prvalue. Together, lvalues and xvalues form glvalues (generalized lvalues); xvalues and prvalues form rvalues.

An lvalue identifies an object with a name or address. A prvalue is a temporary that does not yet have a location; it is used to initialize an object. An xvalue is an expiring value, typically the result of `move` or a function returning an rvalue reference. The xvalue category enables move semantics.

14.2 Move Constructor and Move Assignment

The move constructor steals resources from the source, which must be left in a valid state. A typical move constructor transfers ownership of heap memory, then sets the source's pointer to null:

```
class Buffer {
    char* data_;
    size_t size_;
public:
    Buffer(Buffer&& other) noexcept
        : data_(other.data_), size_(other.size_) {
        other.data_ = nullptr;
        other.size_ = 0;
    }
};
```

The move assignment operator must handle self-assignment and release existing resources before taking ownership:

```
Buffer& operator=(Buffer&& other) noexcept {
    if (this != &other) {
        delete[] data_;
        data_ = other.data_;
        size_ = other.size_;
        other.data_ = nullptr;
        other.size_ = 0;
    }
    return *this;
}
```

14.3 move and forward

`move` unconditionally casts its argument to an rvalue reference. It is a request to treat the object as movable. The actual move happens when the rvalue reference is used to invoke a move constructor or move assignment.

`forward` conditionally casts its argument, preserving its original value category. It is used in forwarding references to perfectly pass arguments to inner functions:

```
template<typename T>
void wrapper(T&& arg) {
    target(forward<T>(arg));
}
```

If `wrapper` is called with an lvalue, `T` deduces to `T&`, and `forward` returns an lvalue reference. If called with an rvalue, `T` deduces to `T`, and `forward` returns an rvalue reference.

14.4 Copy Elision and RVO

Return Value Optimization (RVO) and Named Return Value Optimization (NRVO) allow the compiler to construct the return value directly in the caller's context, skipping copy and move operations. Since C++17, copy elision is mandatory in certain situations (e.g., returning a prvalue). In practice, do not rely on the side effects of copy or move constructors for correctness; they may be elided.

Chapter 15

The Standard Template Library

15.1 Philosophy and Design

The STL is built on three pillars: containers (data structures), algorithms (operations on data), and iterators (glue between containers and algorithms). The design separates data from operations via iterators. An algorithm works on any container that provides suitable iterators, achieving compile-time polymorphism without virtual functions or inheritance.

15.2 Containers Overview

Sequence containers store elements in linear order: `vector`, `deque`, `list`, `forward_list`, `array`. Associative containers store elements in sorted order: `set`, `map`, `multiset`, `multimap`. Unordered associative containers use hash tables: `unordered_set`, `unordered_map`, and their multi variants. Container adaptors provide restricted interfaces: `stack`, `queue`, `priority_queue`.

Each container has different performance characteristics. `vector` is the default choice; it provides contiguous storage and cache-friendly access. Use `list` when frequent insertions and deletions in the middle are needed. Use `map` for sorted key-value access; use `unordered_map` for average O(1) lookup.

15.3 Common Container Operations

Most containers support `size()`, `empty()`, `clear()`, `begin()`, `end()`, `insert()`, `erase()`. C++11 added `emplace` methods that construct elements in place, avoiding copies or moves. Use `emplace_back` instead of `push_back` when constructing with constructor arguments.

15.4 Iterators in Detail

Each container defines its own iterator types: `iterator`, `const_iterator`, `reverse_iterator`, `const_reverse_iterator`. The header `<iterator>` provides `begin`, `end`, and iterator adaptors like `back_inserter`, `inserter`, `ostream_iterator`.

Input iterators can be read once; output iterators can be written once; forward iterators can be read/written multiple times; bidirectional iterators can go backward; random-access iterators support arbitrary jumps. Understanding iterator categories is crucial for writing efficient generic code.

15.5 Algorithms

The `<algorithm>` header provides over 100 algorithms: sorting, searching, partitioning, copying, transforming, removing, and numeric operations. Algorithms operate on iterator ranges `[first, last)`. A range is valid if `last` is reachable from `first`. Passing invalid ranges is undefined behavior.

Use algorithms instead of raw loops whenever possible. They are expressive, less error-prone, and often optimized. Example: sorting a vector:

```
sort(vec.begin(), vec.end());
sort(vec.begin(), vec.end(), greater<>());
```

Parallel algorithms (C++17) accept an execution policy as the first argument: `execution::par` for parallel execution, `execution::par_unseq` for parallel and vectorized, `execution::seq` for sequential. Use them when working with large data sets and when the operations are thread-safe.

15.6 Function Objects and Bind

The `<functional>` header provides predefined function objects (`less`, `greater`, `plus`) and binders. `bind` partially applies arguments to a callable; lambdas are generally preferred for readability. `mem_fn` creates a callable from a member function pointer. `reference_wrapper` created by `ref` allows passing references to function objects that normally take arguments by value.

15.7 Utility Types

`pair<T1, T2>` holds two values. `tuple<T...>` generalizes pair to any number of types. Structured bindings (C++17) decompose pairs, tuples, arrays, and simple structs:

```
auto [name, age] = get_person();
```

`optional<T>` holds either a value of type T or nothing (nullopt). Use it to express that a value may be absent, replacing sentinel values and null pointers.

`variant<T...>` holds one value of a fixed set of types. It is a type-safe union. Use `visit` to apply a visitor to the contained value.

`any` holds a value of any copy-constructible type. It performs type erasure with heap allocation. Use `any_cast` to retrieve the value; a `bad_any_cast` exception is thrown if the type does not match.

`span<T>` (C++20) is a non-owning view of a contiguous sequence. It can refer to an array, vector, or raw buffer, and provides bounds-safe access when desired. It is an ideal function parameter type for sequential data.

15.8 Ranges (C++20)

Ranges elevate iterator pairs to first-class objects. The `<ranges>` header provides range adaptors (views) that lazily transform, filter, and combine ranges. Algorithms in `<algorithm>` now have range-based overloads in `<algorithm>` and range-specific versions in `<ranges>`:

```
auto even = numbers | views::filter([](int n) { return n % 2 == 0; })
                    | views::transform([](int n) { return n * n; });
```

Views are composable, lazy, and do not own the underlying data. They provide a functional, declarative style for processing sequences.

15.9 Chrono

The `<chrono>` library provides type-safe time and duration utilities. Durations are represented as a count of ticks of a specified period. Time points represent an absolute time relative to a clock. Use `chrono::steady_clock` for measuring intervals; `chrono::system_clock` for wall-clock time. Duration casting and arithmetic are safe and explicit.

15.10 Filesystem (C++17)

The `<filesystem>` library provides portable filesystem operations: navigating directories, querying file properties, creating and removing files, and handling paths. `filesystem::path` abstracts path strings. Functions throw `filesystem::filesystem_error` on failure, with overloads accepting `error_code` for non-throwing operation.

Chapter 16

Concurrency

16.1 The Memory Model

C++11 introduced a formal memory model that defines how threads interact through memory. Every variable is a collection of memory locations. A data race occurs when two threads access the same memory location concurrently, at least one is a modification, and there is no synchronization. Data races are undefined behavior. The memory model also defines the semantics of atomics and memory ordering.

16.2 Threads

`thread` represents a single execution thread. Its constructor takes a callable and arguments. The thread begins execution immediately. A thread must be joined or detached before the `thread` object is destroyed; otherwise `terminate` is called. Joining waits for the thread to finish; detaching allows it to run independently.

```
thread t([]() {
    cout << "Running in thread\n";
});
t.join();
```

`jthread` (C++20) automatically joins in its destructor and supports cooperative cancellation via stop tokens. Prefer `jthread` for new code.

16.3 Mutexes and Locks

A mutex (`mutex`, `recursive_mutex`, `shared_mutex`) provides mutual exclusion. Lock the mutex before accessing shared data. Use RAII lock wrappers: `lock_guard`, `unique_lock`, `scoped_lock` (C++17), `shared_lock`.

`lock_guard` is a simple RAII wrapper that locks on construction, unlocks on destruction. `unique_lock` is more flexible, allowing deferred locking, timed locking, and lock transfer. `scoped_lock` locks multiple mutexes without deadlock using a deadlock-avoidance algorithm.

Always lock mutexes in a consistent order. Use `lock` to lock multiple mutexes simultaneously. Never hold a lock while calling user code unless the contract explicitly allows it; this prevents deadlocks from inverted dependencies.

16.4 Condition Variables

`condition_variable` allows threads to wait for a condition to become true. It works with `unique_lock<mutex>`. The waiting thread atomically releases the mutex and blocks; upon notification, it reacquires the mutex before returning. Spurious wakeups are possible, so always wait in a loop checking the condition:

```
unique_lock lock(mutex);
cv.wait(lock, [] { return data_ready; });
```

`condition_variable_any` works with any lock type. Condition variables are fundamental for producer-consumer patterns and thread-safe queues.

16.5 Atomics

`atomic<T>` provides atomic operations on shared variables without locks. It supports load, store, exchange, compare-and-swap, and fetch-and-modify operations. Atomics avoid data races for simple shared data. They can also be used to build lock-free data structures, but such structures are extremely difficult to get right; prefer mutex-based designs unless profiling proves otherwise.

Memory ordering parameters (`memory_order_relaxed`, `memory_order_acquire`, `memory_order_release`, `memory_order_seq_cst`) control the visibility and ordering guarantees. Sequential consistency is the default and the easiest to reason about; relaxed ordering can improve performance but requires deep understanding of the memory model.

16.6 Futures and Promises

`future<T>` represents a value that will be available in the future. `promise<T>` is the writing end: set a value or exception, and the corresponding future becomes ready. `async` launches a task asynchronously and returns a future:

```
auto fut = async(launch::async, [] { return compute(); });
auto result = fut.get();  // blocks if not ready
```

`packaged_task` wraps a callable and provides a future for its result. Futures can be waited on, checked for readiness with `wait_for` and `wait_until`, and shared via `shared_future`.

16.7 Parallel Algorithms (C++17)

As mentioned, the execution policies `execution::seq`, `execution::par`, and `execution::par_unseq` can be passed to many standard algorithms to parallelize their execution. Elements are processed independently, so the operations must not introduce data races. Parallel algorithms can significantly improve performance for large data sets on multicore systems with minimal code changes.

Chapter 17

Compile-Time Programming

17.1 constexpr in Depth

A `constexpr` function is a function that can be evaluated at compile time when given constant arguments. Since C++14, these functions can contain branches, loops, and multiple return statements. Since C++20, they can allocate memory at compile time using `new`, provided all allocations are paired with `delete` before the constant evaluation ends. The allocated memory can also be promoted to static storage using `allocator` or `vector` at compile time.

`constexpr` constructors allow classes to be used at compile time. A `constexpr` object must be initialized with constant expressions and all its member functions that are called must be `constexpr`.

17.2 consteval and constinit

`consteval` functions are immediate functions: they must be evaluated at compile time. Attempting to call them at runtime is a compilation error. They are useful for operations that make no sense at runtime or where compile-time execution is mandatory.

`constinit` asserts that a variable with static storage duration is initialized at compile time. It prevents initialization order problems across translation units (the static initialization order fiasco). Unlike `constexpr`, the variable is not constant.

17.3 Template Metaprogramming

Before `constexpr`, compile-time computation was done via template metaprogramming. Templates are instantiated recursively, and the result is a compile-time constant. For example, computing a factorial:

```
template<int N>
struct Factorial {
    static constexpr int value = N * Factorial<N-1>::value;
};
template<>
struct Factorial<0> {
    static constexpr int value = 1;
};
```

Modern C++ prefers `constexpr` functions over template metaprogramming for compile-time computation because they are easier to write and read. Template metaprogramming is still used for type-level operations, such as type traits and SFINAE.

17.4 Type Traits

The `<type_traits>` header provides a rich set of templates for querying and transforming types at compile time. Traits like `is_integral`, `is_pointer`, `is_same`, `remove_reference`, `enable_if` are building blocks of generic programming.

Since C++17, many type traits have `_v` variable templates (e.g., `is_integral_v<T>`) for cleaner syntax. Use `if constexpr` (C++17) instead of tag dispatch or SFINAE for conditional compilation within a function template:

```
template<typename T>
auto process(T&& val) {
    if constexpr (is_integral_v<T>) {
        return val * 2;
    } else {
        return val + val;
    }
}
```

17.5 SFINAE and Concepts

Substitution Failure Is Not An Error (SFINAE) means that when substituting template parameters fails during overload resolution, the compiler silently discards that candidate instead of emitting an error. This enables selecting overloads based on type properties. SFINAE is a cornerstone of advanced generic programming but produces complex and cryptic code.

Concepts (C++20) replace most SFINAE uses with readable constraints. They provide clear error messages, allow overloading on constraints, and improve the structure of template code.

17.6 CRTP

The Curiously Recurring Template Pattern (CRTP) is an idiom where a class inherits from a base class template instantiated with the derived class as the template argument:

```
template<typename Derived>
class Base {
public:
    void interface() {
        static_cast<Derived*>(this)->implementation();
    }
};

class Derived : public Base<Derived> {
public:
    void implementation() { /* ... */ }
};
```

CRTP provides static polymorphism (compile-time dispatch) as an alternative to virtual functions. It is used in the standard library (`enable_shared_from_this`) and in libraries requiring zero-overhead abstraction.

17.7 Expression Templates

Expression templates delay evaluation of arithmetic expressions to avoid temporary objects. They build a compile-time expression tree representing the computation, which is then evaluated in a single pass. This technique is used in linear algebra libraries like Eigen to achieve high performance. Expression templates rely heavily on template metaprogramming and operator overloading.

Chapter 18

Memory Management Details

18.1 The Stack and the Heap

Automatic variables are allocated on the call stack. The stack is a contiguous region of memory with fast allocation (merely moving a stack pointer) and deterministic deallocation (moving the pointer back on scope exit). Stack size is limited (typically a few megabytes), and deep recursion or large local objects can overflow it.

Dynamic memory is allocated from the heap (free store), a larger but slower region managed by the runtime allocator. Allocation and deallocation are expensive operations, and fragmentation can degrade performance. The RAII idiom and smart pointers manage heap objects safely.

18.2 Alignment

Every object type has an alignment requirement, typically equal to its size for fundamental types. `alignof(T)` returns the alignment. The compiler inserts padding between struct members and at the end to satisfy alignment. Misaligned access can crash on some architectures and degrade performance on others. Use `alignas` to specify stricter alignment when needed (e.g., for SIMD types).

18.3 Memory Layout of Objects

The memory layout of an object includes its non-static data members, possibly with padding, and, for polymorphic classes, a hidden vtable pointer. The exact layout is implementation-defined. Understanding layout is important for serialization, interfacing with C code, and performance optimization.

18.4 Placement new and Custom Allocators

Placement `new` constructs an object in a pre-allocated buffer. This is the foundation of custom allocators and container memory management. The standard library containers take an allocator template parameter, defaulting to `allocator<T>`. Custom allocators can pool memory, track allocations, or use special memory regions.

18.5 Undefined Behavior

Undefined behavior (UB) means the standard imposes no requirements on the program's behavior. The compiler may assume UB never happens and optimize accordingly. Common sources of UB: signed integer overflow, dereferencing null or dangling pointers, out-of-bounds array access, data races, use of uninitialized variables. UB is not a runtime error with a friendly message; it is a license for the compiler to generate code that does unexpected things, including executing code paths that appear unreachable. Tools like UBSan (Undefined Behavior Sanitizer) detect many forms of UB at runtime.

Chapter 19

Input/Output

19.1 Streams Overview

The iostream library provides a hierarchy of stream classes for formatted and unformatted I/O. `cin` is the standard input, `cout` the standard output, `cerr` for error output (unbuffered), and `clog` for logging (buffered). Streams work with the insertion («) and extraction (») operators.

19.2 File I/O

`ifstream`, `ofstream`, and `fstream` provide file-based streams. They are RAII objects: opening the file in the constructor and closing in the destructor. Check the stream state after operations using `!file` or file state flags: `good()`, `eof()`, `fail()`, `bad()`.

```
ifstream input("data.txt");
if (!input) {
    cerr << "Failed to open file.\n";
    return;
}
string line;
while (getline(input, line)) {
    process(line);
}
```

19.3 String Streams

`stringstream`, `istringstream`, `ostringstream` allow reading from and writing to strings as if they were streams. They are useful for formatting, parsing, and building strings without manual concatenation.

19.4 Stream Formatting

Manipulators like `setw`, `setprecision`, `fixed`, `hex` control formatting. Format flags can be set via member functions. Since C++20, `format` (from the `<format>` library) provides a safer, more readable alternative to iostream for many formatting tasks, inspired by Python's `str.format`:

```
string s = format("Hello, {}! The answer is {}.", name, 42);
```

Chapter 20

Modules (C++20)

Modules replace the textual inclusion model of header files with a structured, compiler-managed interface. A module interface file declares what is exported; implementation files import what they need. Modules improve compile times, reduce macro pollution, and enforce isolation.

```
// math.ixx (module interface)
export module math;
export int add(int a, int b) { return a + b; }

// main.cpp
import math;
int main() { return add(2, 3); }
```

Module support requires build system and compiler support. As of C++20, the specification is complete, but adoption is ongoing. Modules are the future of C++ project structure.

Chapter 21

Build Systems and Packaging

21.1 CMake Essentials

CMake is a meta-build system. It reads `CMakeLists.txt` files and generates native build files. The core commands:

- `cmake_minimum_required(VERSION ...)`
- `project(Name LANGUAGES CXX)`
- `set(CMAKE_CXX_STANDARD 20)`
- `add_executable(name sources...)`
- `add_library(name STATIC|SHARED sources...)`
- `target_include_directories(name PUBLIC|PRIVATE dirs)`
- `target_link_libraries(name PUBLIC|PRIVATE libraries)`
- `find_package(Name REQUIRED)`
- `target_compile_options(name PRIVATE flags)`

Modern CMake (3.0+) emphasizes targets and properties over global variables. Use `PUBLIC`, `PRIVATE`, and `INTERFACE` keywords to specify how dependencies propagate. Install rules, testing with CTest, and packaging with CPack are integrated.

21.2 Static and Shared Libraries

A static library (`.a` on Unix, `.lib` on Windows) is an archive of object files linked into the executable at link time. The code is duplicated in each executable. A shared library (`.so`, `.dylib`, `.dll`) is loaded at load time or runtime and shared among multiple executables. Shared libraries reduce disk and memory footprint but introduce ABI compatibility concerns.

When building shared libraries, symbols must be exported (`__declspec(dllexport)` on Windows, visibility attributes on Unix). C++ name mangling and ABI incompatibilities between compiler versions make C interfaces (`extern "C"`) the stable boundary for shared libraries.

21.3 Linking and ABI

The Application Binary Interface (ABI) includes name mangling, virtual table layout, struct layout, calling conventions, and exception handling mechanics. ABI is not standardized across compilers or even across versions of the same compiler. When distributing libraries, provide source or use a stable C ABI. The C++ standard defines language semantics, not binary layout. This is why header-only libraries are popular in C++: they avoid ABI issues by compiling everything together with the same settings.

Chapter 22

Coding Standards and Best Practices

22.1 The C++ Core Guidelines

The C++ Core Guidelines, edited by Bjarne Stroustrup and Herb Sutter, provide rules for writing modern, safe C++. Key principles: prefer scoped objects, avoid raw new and delete, use RAII, use algorithms instead of raw loops, prefer compile-time checks over runtime checks, and write clear, simple code.

22.2 Naming Conventions

Consistency within a project is more important than any particular style. Common conventions:

- Class names: PascalCase (`MyClass`)
- Functions and variables: camelCase or snake_case (`doWork`, `do_work`)
- Constants: kCamelCase or ALL_CAPS
- Private members: trailing underscore (`value_`) or leading underscore (reserved to the implementation in global namespace)
- Macros: ALL_CAPS with underscores

Avoid single-letter names except for loop indices; use meaningful, self-documenting names.

22.3 Common Beginner Mistakes

- Using `using namespace std;` in headers or globally, causing name collisions.
- Returning references or pointers to local variables (dangling references).
- Forgetting the virtual destructor in polymorphic base classes.
- Mixing `new[]` with `delete` (and vice versa), or using raw new/delete at all.
- Accessing moved-from objects without reinitializing them.
- Catching exceptions by value, causing slicing.
- Assuming `vector<bool>` behaves like a normal vector; it is a packed specialization.
- Ignoring compiler warnings.
- Using macros when constexpr, inline, or templates would work.
- Passing large objects by value instead of const reference.

22.4 Performance Guidelines

- Profile before optimizing; guesswork is unreliable.
- Use `vector` as the default container.
- Pre-allocate with `reserve()` when size is known.
- Avoid unnecessary copies; pass by const reference or use move semantics.
- Use `emplace` methods to construct in place.
- Minimize memory allocations in hot loops.
- Use contiguous containers for cache locality.
- Enable compiler optimizations (`-O2`, `-O3`) in release builds.
- Use link-time optimization (`-flto`) for cross-translation-unit inlining.
- Be aware of branch prediction and avoid unpredictable branches in performance-critical code.

22.5 Security Considerations

- Never use `gets()` or unbounded string input.
- Prefer `string` and `vector` over C arrays.
- Use bounds-checked access (`at()`) or span where appropriate.
- Sanitize all external input.
- Avoid format string vulnerabilities; use `format`.
- Be aware of integer overflow and signedness issues.
- Minimize use of raw pointers and manual memory management.
- Use smart pointers to express ownership semantics.

Chapter 23

Advanced Patterns and Idioms

23.1 RAII in Practice

Resource Acquisition Is Initialization is the most important idiom in C++. Acquire resources in a constructor, release in the destructor. This applies to memory, file handles, locks, sockets, and any other resource that must be released. The standard library provides RAII wrappers: `unique_ptr`, `lock_guard`, `ifstream`. Write your own RAII classes for non-standard resources; they should be non-copyable unless shared ownership is intended, and their destructors should not throw.

23.2 PIMPL

The Pointer to Implementation (PIMPL) idiom hides implementation details from the class's public header. The class holds a pointer (typically `unique_ptr`) to an implementation struct defined in the source file. This reduces compile-time dependencies and preserves ABI stability when the implementation changes.

```
// widget.hpp
class Widget {
public:
    Widget();
    ~Widget();
    void do_something();
private:
    struct Impl;
    unique_ptr<Impl> pImpl;
};

// widget.cpp
struct Widget::Impl { /* actual members */ };
Widget::Widget() : pImpl(make_unique<Impl>()) {}
Widget::~Widget() = default;  // must be defined after Impl is
   complete
void Widget::do_something() { pImpl->do_something(); }
```

23.3 Singleton

The Singleton pattern ensures a class has exactly one instance. In C++, the canonical implementation is a function-local static variable:

```
class Singleton {
public:
    static Singleton& instance() {
        static Singleton inst;
        return inst;
    }
private:
    Singleton() = default;
```

```
    Singleton(const Singleton&) = delete;
    void operator=(const Singleton&) = delete;
};
```

This is thread-safe since C++11, when the standard guarantees that local static initialization is performed exactly once, with concurrent callers blocked until initialization completes. Singletons introduce global state and should be used sparingly.

23.4 Factory Pattern

Factories separate object creation from usage. A factory function returns a base pointer or `unique_ptr` to a derived type, allowing the client to work polymorphically without knowing the concrete type. Use templates for compile-time factories and virtual functions for runtime dispatch.

23.5 Policy-Based Design

Policy-based design uses templates to compose behavior from small, independent policy classes. Each policy defines a specific aspect of the algorithm or class. The host class template takes policies as template parameters and defers to them for specific decisions. This achieves extreme flexibility with zero runtime overhead. The standard library's allocator parameter is an example of a policy. This pattern is heavily used in modern C++ libraries.

Chapter 24

Testing and Debugging

24.1 Assertions

`assert()` from `<cassert>` checks that a condition holds at runtime. If it fails, the program terminates with a diagnostic message. Assertions are typically enabled in debug builds and disabled in release by defining `NDEBUG`. Use assertions to document and enforce invariants that must always be true. For errors that can occur in normal operation (e.g., invalid user input), use exceptions or error codes, not assertions.

`static_assert` (C++11) performs compile-time checks. It takes a constant expression and an optional string message. Use it to enforce template constraints, verify type sizes, and check platform assumptions.

24.2 Sanitizers

Compiler sanitizers instrument the program to detect runtime errors. AddressSanitizer (ASan) detects memory errors: out-of-bounds access, use-after-free, double-free, memory leaks. UndefinedBehaviorSanitizer (UBSan) detects signed overflow, null pointer dereference, invalid casts, and other UB. ThreadSanitizer (TSan) detects data races. Compile with `-fsanitize=address` or `-fsanitize=undefined` and run tests. Sanitizers incur runtime overhead but are invaluable for catching bugs early.

24.3 Static Analysis

Static analysis tools examine source code without executing it. Compiler warnings (`-Wall`, `-Wextra`, `-Wpedantic`) are a basic form. Clang-Tidy provides hundreds of checks for modern C++ idioms, performance, and style. Cppcheck is a standalone analyzer. Integrate static analysis into your CI pipeline to catch issues before code review.

24.4 Debugging

A debugger (GDB, LLDB, Visual Studio Debugger) allows stepping through code, inspecting variables, and examining memory. Core dumps can be analyzed post-mortem. Debug builds (`-g`, no optimization) provide full symbolic information. For optimized code, debugging is harder but possible with some experience.

24.5 Profiling

Profilers measure where a program spends time and memory. Sampling profilers (perf on Linux, Instruments on macOS, Visual Studio Profiler) provide statistical overviews. Instrumenting profilers provide precise call counts. Benchmark micro-optimizations with a framework like Google Benchmark; always measure, because intuition about performance is often wrong. Cache misses, branch mispredictions, and memory allocation patterns are often more important than instruction count.

Chapter 25

C++ and the Real World

25.1 When to Use C++

Choose C++ when performance is critical: game engines, real-time systems, high-throughput servers, embedded devices with limited resources. Choose it when you need direct hardware control or when you are building on a large existing C++ codebase. Choose it when you need predictable resource management, including deterministic destruction and the absence of a garbage collector pause.

25.2 When Not to Use C++

C++ is not the best choice for rapid prototyping where development speed trumps performance; Python or JavaScript may be better. It is not ideal for string-processing-heavy scripts; the string manipulation syntax is more verbose than in Perl or Python. For small, short-lived programs that will not be maintained long-term, the complexity of C++ may not be justified. For memory-safe applications where performance is secondary, languages like Rust or managed languages like C# and Java provide stronger safety guarantees at a lower cognitive load.

25.3 Staying Current

C++ continues to evolve. Follow the ISO C++ committee papers, attend conferences (CppCon, C++Now, Meeting C++), and read blogs from the community. The C++ Core Guidelines and cppreference.com are essential references. Practice writing modern C++ even in existing codebases by gradually adopting new features during maintenance.

25.4 Final Words

C++ is a language of unparalleled power and depth. It rewards study and practice with the ability to build systems that are fast, efficient, and expressive. It punishes carelessness with subtle bugs and long debugging sessions. The path to mastery is long, but each new concept builds on previous ones. Write code. Read code. Learn from mistakes. The language tools and idioms presented in this book are your foundation; real skill comes from applying them to real problems.